

# **EXCEPTION HANDLING + TRICKY OOP**

**Q1. What is an exception?**

- a) Syntax error
- b) Runtime abnormal condition
- c) Compile-time error
- d) Logical error

**Answer: b**

---

**Q2. Which keyword is used to handle exceptions?**

- a) catch
- b) try
- c) throw
- d) All of the above

**Answer: d**

---

**Q3. Which block must come first?**

- a) catch
- b) finally
- c) try
- d) throw

**Answer: c**

---

**Q4. What is the superclass of all exceptions?**

- a) Exception
- b) Throwable
- c) Error
- d) RuntimeException

**Answer: b**

---

**Q5. Checked exceptions are checked at?**

- a) Runtime
- b) Compile-time
- c) Both
- d) Never

**Answer: b**

---

**Q6. Which is unchecked exception?**

- a) IOException
- b) SQLException
- c) NullPointerException
- d) FileNotFoundException

**Answer:** c

---

**Q7. What will happen if exception is not handled?**

- a) Ignored
- b) Program terminates
- c) Auto-corrected
- d) Compiles fine

**Answer:** b

---

**Q8. What is finally block used for?**

- a) Catch exception
- b) Always executed
- c) Define exception
- d) Throw exception

**Answer:** b

---

**Q9. Can finally block be skipped?**

- a) Yes
- b) No
- c) Only in error
- d) Only in runtime

**Answer:** a

👉 (Skipped if JVM exits like `System.exit()`)

---

**Q10. What will be output?**

```
public class Test {  
    public static void main(String[] args) {  
        try {  
            int x = 10/0;  
        } catch (ArithmeticException e) {  
            System.out.println("Exception");  
        } finally {  
            System.out.println("Finally");  
        }  
    }  
}
```

- a) Exception
- b) Finally
- c) Exception Finally
- d) Error

**Answer:** c

---

**Q11. What will be output?**

```
public class Test {  
    public static void main(String[] args) {  
        try {  
            int x = 10/0;  
        } finally {  
            System.out.println("Finally");  
        }  
    }  
}
```

- a) Finally
- b) Exception Finally
- c) Runtime error
- d) Compile error

**Answer:** b

---

**Q12. Which keyword is used to explicitly throw exception?**

- a) throws
- b) throw
- c) catch
- d) final

**Answer:** b

---

**Q13. throws keyword is used for?**

- a) Handling exception
- b) Declaring exception
- c) Ignoring exception
- d) Catching exception

**Answer:** b

---

**Q14. Can we have multiple catch blocks?**

- a) Yes
- b) No
- c) Only two

d) Only one

**Answer: a**

---

**Q15. Order of catch blocks should be?**

a) General → Specific

b) Specific → General

c) Any order

d) Alphabetical

**Answer: b**

---

### ◆ Output-Based Exception Questions

---

**Q16. What will be output?**

```
public class Test {  
    public static void main(String[] args) {  
        try {  
            System.out.println("Try");  
            return;  
        } finally {  
            System.out.println("Finally");  
        }  
    }  
}
```

a) Try

b) Finally

c) Try Finally

d) Nothing

**Answer: c**

👉 Finally executes even after return

---

**Q17. What will be output?**

```
public class Test {  
    public static void main(String[] args) {  
        try {  
            System.out.println(10/0);  
        } catch (Exception e) {  
            System.out.println("Caught");  
        }  
    }  
}
```

- a) Error
- b) Caught
- c) 0
- d) Exception

**Answer:** b

---

**Q18. What will be output?**

```
public class Test {  
    public static void main(String[] args) {  
        try {  
            int x = 10;  
        } catch (Exception e) {  
            System.out.println("Catch");  
        }  
    }  
}
```

- a) Catch
- b) Error
- c) Nothing
- d) Runtime error

**Answer:** c

---

**Q19. What will be output?**

```
public class Test {  
    public static void main(String[] args) {  
        try {  
            int x = 10/0;  
        } catch (NullPointerException e) {  
            System.out.println("Caught");  
        }  
    }  
}
```

- a) Caught
- b) Error
- c) Nothing
- d) Compile error

**Answer:** b

👉 Wrong catch type

---

**Q20. What will be output?**

```
public class Test {  
    public static void main(String[] args) {  
        try {  
            int x = 10/0;  
        } catch (ArithmeticException e) {  
            System.out.println("A");  
        } catch (Exception e) {  
            System.out.println("B");  
        }  
    }  
}
```

- a) A
- b) B
- c) Error
- d) Nothing

**Answer: a**

---

#### ◆ Tricky OOP + Mixed Concepts

---

**Q21. What will be output?**

```
class Test {  
    Test() {  
        System.out.println("Constructor");  
    }  
    public static void main(String[] args) {  
        Test t = null;  
        t = new Test();  
    }  
}
```

- a) Constructor
- b) Error
- c) Null
- d) Nothing

**Answer: a**

---

**Q22. What happens when object reference is null and method is called?**

- a) Compile error
- b) Runtime error
- c) NullPointerException
- d) Nothing

**Answer: c**

---

**Q23. What will be output?**

```
class Test {  
    static int x = 10;  
    public static void main(String[] args) {  
        Test t1 = new Test();  
        Test t2 = new Test();  
        t1.x = 20;  
        System.out.println(t2.x);  
    }  
}
```

- a) 10
- b) 20
- c) Error
- d) Null

**Answer: b**

👉 Static shared across objects

---

**Q24. What will be output?**

```
class Test {  
    int x;  
    public static void main(String[] args) {  
        Test t = new Test();  
        System.out.println(t.x);  
    }  
}
```

- a) 0
- b) Null
- c) Error
- d) Garbage

**Answer: a**

---

**Q25. What will be output?**

```
class Test {  
    void show() {  
        System.out.println("A");  
    }  
}  
class Demo extends Test {  
    void show() {  
        System.out.println("B");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Test t = new Demo();  
        t.show();  
    }  
}
```

- a) A
- b) B
- c) Error
- d) Null

**Answer: b**

---

#### Q26. What will be output?

```
class Test {  
    final int x = 10;  
    void change() {  
        x = 20;  
    }  
}
```

- a) 20
- b) 10
- c) Compile error
- d) Runtime error

**Answer: c**

---

#### Q27. What will be output?

```
class Test {  
    static void show() {  
        System.out.println("A");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Test t = null;  
        t.show();  
    }  
}
```

- a) A
- b) Error
- c) NullPointerException
- d) Nothing

**Answer: a**



👉 Static call via null reference

---

**Q28. What is marker interface?**

- a) Interface with methods
- b) Empty interface
- c) Abstract class
- d) Static class

**Answer: b**

---

**Q29. What will be output?**

```
class Test {  
    int x;  
    Test() {  
        x = 5;  
    }  
    public static void main(String[] args) {  
        Test t = new Test();  
        System.out.println(t.x);  
    }  
}
```

- a) 0
- b) 5
- c) Error
- d) Null

**Answer: b**

---

**Q30. What will be output?**

```
class Test {  
    int x = 10;  
    public static void main(String[] args) {  
        Test t = new Test();  
        System.out.println(t);  
    }  
}
```

- a) 10
- b) Error
- c) ClassName@hashcode
- d) Null

**Answer: c**